



CSC 1012 - Introduction to Computer Programming

Smart Hospital & Resource Allocation System - Project Report

Name: Samathi Abeynayake

Student ID: AS20250424



Contents

1.High-Level Architecture	3
2. List of Implemented Functions.....	3
3. Data Structure Design	5
3.1 Lookup Tables (Requirement 1).....	5
3.2 Bed Occupancy Matrix.....	5
3.3 Patient Records (Requirement 2)	5
3.4 Queue Tracking	5
3.5 Persistent Patient ID Counter.....	5
4. GitHub Repository	6
5. Assumptions Made During Development	6
6. Sample Output	7



1.High-Level Architecture

The system is a menu-driven console application written in C, split across three files to separate concerns:

- main.c – program entry point; runs the menu loop and dispatches to the relevant function based on the user's choice.
- hospital.h – all constants, lookup-table declarations, patient data declarations, and function prototypes shared across files.
- hospital.c – all lookup data, patient records, billing calculations, sorting, reporting, and file persistence logic.

The program flow is a do-while loop that repeatedly displays a 5-option menu (Register Patient, View Bed Occupancy, View Patients by Priority, Generate Reports, Save & Exit) and calls the corresponding function until the user chooses to exit. Bed occupancy is loaded from disk at startup and saved back after every registration and on exit, so hospital state persists across runs.

2. List of Implemented Functions

Function	File	Purpose
getValidatedInt()	hospital.c	Re-prompts until a whole number within a given range is entered; clears bad input from the buffer
showMainMenu()	hospital.c	Prints the 5-option main menu
registerPatient()	hospital.c	Collects patient details, runs all billing calculations, assigns a bed, logs the record
assignBed()	hospital.c	Finds the first free bed in a ward's row of the occupancy matrix and marks it occupied
calculateWaitTime()	hospital.c	Wait Time = specialty queue count × average consultation time
calculateSurcharge()	hospital.c	Emergency surcharge: 0% / 20% / 50% of base fee by triage level
calculateWardCost()	hospital.c	Ward Stay Cost = Days Admitted × Ward Daily Rate
calculateGrossTotal()	hospital.c	Gross Total = Base Fee + Surcharge + Ward Cost
calculateDiscount()	hospital.c	15% age-subsidy discount for ages under 5 or over 65
calculateFinalAmount()	hospital.c	Final Payable = Gross Total – Discount
displayBill()	hospital.c	Prints the formatted admission & bill summary for one patient
viewBedOccupancy()	hospital.c	Displays the bed occupancy matrix and occupancy % per ward
sortPatientsByPriority()	hospital.c	Stable bubble sort of patients by descending triage level
viewSortedPatients()	hospital.c	Displays all patients ordered by triage priority
generateReports()	hospital.c	Summary analytics: counts by urgency, revenue, discounts, bed occupancy %, top payer



loadBedStatus() / saveBedStatus()	hospital.c	Persists the bed occupancy matrix to beds_status.txt between runs
loadNextPatientId() / saveNextPatientId()	hospital.c	Persists the next patient ID counter to next_id.txt so IDs never repeat across separate program runs
appendPatientRecord()	hospital.c	Appends a permanent billing log line to patient_records.txt
main()	main.c	Menu-driven program loop dispatching to the functions above



3. Data Structure Design

3.1 Lookup Tables (Requirement 1)

Specialty and ward information is stored in constant parallel arrays, indexed 0–3 (representing Specialty/Ward IDs 1–4): `specialtyName[]`, `specialtyBaseFee[]`, `specialtyConsultTime[]`, `specialtyDailyCap[]` for specialties, and `wardName[]`, `wardDailyRate[]`, `wardCapacity[]` for wards.

3.2 Bed Occupancy Matrix

A 2D integer array, `bedOccupancy[NUM_WARDS][MAX_BEDS_PER_WARD]`, tracks bed status per ward (0 = Available, 1 = Occupied). Only the first `wardCapacity[w]` entries of each row are used, since ward capacities differ (5–20 beds).

3.3 Patient Records (Requirement 2)

Patient data is stored using parallel arrays indexed by patient number (0-based), as recommended in the assignment brief, rather than structs. Each patient's data – name, age, urgency, specialty index, admission status, ward index, bed number, days admitted, and every calculated billing field – lives at the same index *i* across all of the following arrays: `patientName[]`, `patientAge[]`, `patientUrgency[]`, `patientSpecialtyIdx[]`, `patientIsAdmitted[]`, `patientWardIdx[]`, `patientBedNumber[]`, `patientDaysAdmitted[]`, `patientBaseFee[]`, `patientSurcharge[]`, `patientWardCost[]`, `patientGrossTotal[]`, `patientDiscount[]`, `patientFinalAmount[]`, and `patientWaitTime[]`.

3.4 Queue Tracking

`specialtyQueueCount[NUM_SPECIALTIES]` tracks how many patients have queued for each specialty so far, incremented after each registration and used to calculate the next patient's estimated wait time.

3.5 Persistent Patient ID Counter

`nextPatientId` is a single integer persisted to `next_id.txt`, loaded at program startup and saved immediately after each new patient is assigned an ID. This ensures Patient IDs keep incrementing (PAT-1001, PAT-1002, ...) across separate program runs instead of restarting at PAT-1001 every time the program is launched, which would otherwise create duplicate IDs in the permanent `patient_records.txt` log. Each patient's assigned ID is stored in the parallel array `patientID[]`, since it can no longer be derived purely from the in-memory array index once persistence is involved.



4. GitHub Repository

Repository URL:

<https://github.com/SamathiAbeynayake05/SmartHospitalSystem>

The repository contains the full commit history from project start, with each commit representing one incremental feature (lookup data, patient arrays, registration, billing calculations, display, sorting, reporting, file persistence, and bug fixes).

5. Assumptions Made During Development

- Patient IDs are generated from a persistent counter stored in next_id.txt, starting at PAT-1001. The counter loads at startup and saves after every registration, so IDs keep incrementing across separate program runs and never repeat – important since patient_records.txt is meant to be a permanent log across sessions, not just one run.
- If a chosen ward is at full capacity, the patient is automatically registered as an outpatient instead of blocking registration entirely.
- Estimated wait time uses the specialty's queue count from before the current patient is added, so the first patient registered for any specialty always has a wait time of 0 minutes.
- Bed numbers are displayed as 1-indexed (e.g. “Bed #01”) for readability, even though the underlying array is 0-indexed.
- MAX_PATIENTS is capped at 100 for the parallel array sizes, which comfortably covers a single testing/demo session.
- All numeric user input is validated with a re-prompt loop (getValidatedInt()); non-numeric or out-of-range entries are rejected with an error message rather than crashing the program.
- A specialty reaching its daily patient cap (specialtyDailyCap[]) produces a warning note but does not block further registration for that specialty, since the brief does not require hard enforcement – only that the cap be tracked.



6. Sample Output

The program's bill printout was verified against the sample output given in the assignment brief (Ms Samathi Abeynayake, age 20, Cardiology, ICU, 2 days, Critical) and produces a matching Final Payable Amount of LKR 56750.

```
"C:\Users\Samathi Abeynayak" x + v
Smart Hospital Patient & Resource Allocation System
Bed status loaded from beds_status.txt

===== SMART HOSPITAL SYSTEM =====
1. Register New Patient
2. View Bed Occupancy Matrix
3. View Patients by Priority (Triage Sort)
4. Generate Performance Reports
5. Save & Exit
=====
Enter your choice (1-5): 1

--- New Patient Registration ---
Patient Name: Samathi Abeynayake
Patient Age (0-120): 20
Triage Level (1 = Normal, 2 = Urgent, 3 = Critical): 3

Available Specialties:
1. General Practice (OPD) (LKR 1500.00)
2. Paediatrics (LKR 2500.00)
3. Cardiology (LKR 4500.00)
4. Neurology (LKR 5000.00)
Select Specialty ID (1-4): 3
Is Admitted to Ward? (1 = Yes, 0 = No): 1
```



"C:\Users\Samathi Abeynayak X + v

Available Wards:
1. General Ward (LKR 3000.00/day)
2. Paediatric Ward (LKR 6000.00/day)
3. Surgical Ward (LKR 12000.00/day)
4. ICU (Intensive Care Unit) (LKR 25000.00/day)
Select Ward ID (1-4): 4
Days Admitted (1-365): 2

=====

SMART HOSPITAL ADMISSION & BILL

Patient ID : PAT-1001
Patient Name : Samathi Abeynayake
Age : 20 Years
Specialty : Cardiology
Assigned Ward : ICU (Intensive Care Unit) (Bed #01)
Urgency Level : Level 3 (Critical)

Base Consultation Fee : LKR 4500.00
Emergency Surcharge : LKR 2250.00 (50%)
Ward Stay Cost (2 Days) : LKR 50000.00

Gross Total Bill : LKR 56750.00
Age Subsidy Discount : LKR 0.00

Final Payable Amount : LKR 56750.00
Estimated Waiting Time: 0.00 mins (Immediate Attention)

=====